Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la Technologie

Université de Sfax | Faculté des Sciences de Sfax

Département d'Informatique | Master MRSI

An Agentic Framework for Infrastructure as Code Misconfiguration Analysis and Remediation

Technical Report — Rapport 4

Technology Positioning, Empirical Plan, and Repositioned Contributions

Prepared by

Ahmedou Yahye Kheyri

Supervised by

Pr. Hala Bezine

Keywords: Agentic Framework • Infrastructure as Code • Security Smells • Retrieval-Augmented Generation • Patch Validation • Large Language Models

Academic Year 2025–2026 | April 2026

Abstract

This fourth technical report consolidates the design (Rapport 2), the corpus (Rapport 3), and the internal critique (Rapport-Critique) of the thesis into a single coherent positioning of the work as of April 2026. Two changes relative to Rapport 2 are central. First, the project is repositioned around the empirical contribution that has actually materialised: a 33,667-record Infrastructure as Code (IaC) security fix corpus, scraped, classified by a 44-entry smell taxonomy, and undergoing dynamic validation by Checkov [1]. Second, the contribution is reframed from a generic "agentic framework for IaC security" into a specifically positioned artefact within a 2025–2026 benchmark landscape: *agentic IaC remediation with independent scanner-based validation*. A landscape grid is constructed across PatchEval [2], AutoPatchBench [3], SEC-bench [4], SecRepoBench [5], IaC-Eval [6], Multi-IaC-Eval [7], IntelliSA [8], SecLLM [9], and GenSIAC [10], and the target cell is shown to be unoccupied. The framework architecture is carried over from Rapport 2 with two amendments: KICS [11] is added as a second validator to extend coverage to Ansible, Kubernetes, Dockerfile, and CloudFormation, and the log-prob-based confidence score is replaced by a self-consistency surrogate. The evaluation methodology is reduced from the previously claimed eighteen metrics to a defensible eight, with Patch Validity Rate (PVR), Smell Elimination Rate (SER), and No-New-Issues Rate (NNIR) retained as the primary outcomes and the Wilcoxon rank-sum analysis kept as the inferential test. A planned experimental protocol is given for Configurations B, C, and D on a stratified 3,371-record held-out test split, with IntelliSA [8] fixed as the detection-side State of the Art (SOTA) baseline. Threats to validity are stated explicitly and an updated implementation timeline is given.

Keywords: Agentic Framework, Infrastructure as Code, Security Smells, Retrieval-Augmented Generation, Patch Validation, Large Language Models, Empirical Software Engineering.

Contents

Abstract **1**

List of Acronyms **7**

1 Technology Timeline and Positioning **8**

1.1 Why this chapter exists 8

1.2 Technology evolution, 2019–2026 8

1.3 Landscape grid: task × domain 11

1.4 Implications for the rest of this report 11

2 Updated State of the Art **13**

2.1 New IaC detection baseline: IntelliSA 13

2.2 Methodological lineage of PVR: PatchEval and AutoPatchBench 13

2.3 Benchmark-construction precedent: SEC-bench and SecRepoBench . . 14

2.4 IaC generation calibration: IaC-Eval and Multi-IaC-Eval 14

2.5 Survey context: agentic Retrieval-Augmented Generation (RAG) in 2025 15

2.6 Carry-over of unchanged sections 15

3 Framework Architecture (Carried Over with Two Amendments) **16**

3.1 Reference architecture (recap) 16

3.2 Amendment 1: KICS as second validator 17

3.2.1 Why 17

3.2.2 What changes 17

3.2.3 Where in the code 17

3.3 Amendment 2: self-consistency confidence 17

3.3.1 Why 17

3.3.2 What changes 18

3.3.3 Where in the code 18

3.4 Clarification on input/output contracts 18

4 Dataset (33,667-record corpus, promoted to primary) 19

4.1 From a sanity-check suite to a primary dataset 19

4.2 Corpus composition 20

4.2.1 Smell distribution 20

4.2.2 Tiering 20

4.3 Train / validation / test split 21

4.4 Comparison to Multi-IaC-Eval 22

4.5 Datasets used as auxiliary inputs and baselines 22

5 Evaluation Methodology (Revised) 24

5.1 What changes from Rapport 2 24

5.2 Definitions 26

5.3 Configurations to evaluate 27

5.4 What was dropped from Rapport 2 27

6 Experimental Plan and Reproducibility 28

6.1 Configuration A: baseline reproduction 28

6.2 Configurations B, C, D: protocol 28

6.2.1 Inputs and outputs 28

6.2.2 Compute and runtime budget 28

6.2.3 Reproducibility checklist 29

6.2.4 Headline tables to be filled in 29

6.3 What success looks like 29

7 Threats to Validity 31

7.1 Internal validity 31

7.2 External validity 31

7.3 Construct validity 32

8 Conclusion and Perspectives 33

8.1 What this report establishes 33

8.2 What remains for Rapport 5 33

8.3 Long-term perspectives 34

A Eight-file sanity suite (carried from Rapport 2) 38

B Reproducibility manifest schema 39

List of Figures

1.1 Technology-evolution timeline across four tracks: agentic RAG, IaC smell detection, benchmarks, and IaC taxonomies. The thesis target sits at the intersection of all four: a remediation contribution that uses the 2024-era Corrective Retrieval-Augmented Generation (CRAG) retry pattern, the 2025 War 62-category taxonomy, and the 2025 PatchEval-style scanner-validated evaluation, applied to the IaC domain where IntelliSA [8] now sets the 2026 detection baseline. 8

3.1 Reference architecture (carried over from Rapport 2). The two changes in this report are: (i) the External Tool Integrator now runs KICS in addition to Checkov, and (ii) the Fix Generator emits a self-consistency confidence score over $N = 5$ samples instead of a log-prob. 16

List of Tables

1.1 Milestone list backing Figure 1.1. 10

1.2 Benchmark landscape, April 2025 – April 2026. 11

4.1 Corpus composition by IaC tool, after the salvage pass of Rapport 3 §2.4. Counts may shift modestly after the Checkov validator finishes the full sweep (Phase 4 of `Code/scraping/plan.md`). 20

4.2 This thesis's corpus vs. Multi-IaC-Eval [7]. 22

5.1 Revised evaluation metric set used by Rapport 4 and downstream. . . . 25

5.2 The four configurations of the framework. Rapport 2 only ran Configuration A (8-file suite, no RAG, no validator). Rapport 4 plans Configurations B, C, and D on the 33k corpus's test split. 27

6.1 Headline results, to be filled in once Configurations B/C/D are run. Metrics defined in Chapter 5. 29

List of Acronyms

API Application Programming Interface. 16, 19, 26, 28, 31

CDK Cloud Development Kit. 10, 13, 21

CFN AWS CloudFormation. 10, 13, 16, 20, 21, 30

CRAG Corrective Retrieval-Augmented Generation. 5, 7, 12, 14

CVE Common Vulnerabilities and Exposures. 9, 10, 12, 13

CWE Common Weakness Enumeration. 9, 10, 12

IaC Infrastructure as Code. 1, 2, 5–7, 9–14, 18, 19, 30, 33

LLM Large Language Model. 7, 9, 10, 12, 13, 17, 21, 27, 30, 31, 33

NNIR No-New-Issues Rate. 1, 11, 24–26, 28, 29, 31, 32

PVR Patch Validity Rate. 1, 2, 9–14, 16, 22, 24–26, 28, 29, 31–33

RAG Retrieval-Augmented Generation. 2, 5–7, 9, 12, 14, 26, 27, 29, 33

SER Smell Elimination Rate. 1, 11, 24–26, 28, 32

SOTA State of the Art. 1, 7, 9, 28

Chapter 1

Technology Timeline and Positioning

1.1 Why this chapter exists

Rapport 2 introduced the framework on the strength of three references whose most recent publication was 2024 and whose detection baseline was an Large Language Model (LLM)-only system (SecLLM, GenSIAC, War et al.). Between writing Rapport 2 and writing this report, the relevant literature has moved: a new SOTA on IaC smell detection (IntelliSA, January 2026), a patch-validation benchmark with the same dynamic-validation philosophy as this thesis (PatchEval, November 2025), and a multi-format IaC mutation benchmark (Multi-IaC-Eval, September 2025) have all appeared. A repositioning is therefore required *before* reporting any further experiments.

This chapter rebuilds the temporal context, then collapses it into a two-dimensional landscape grid (task × domain) that locates the thesis contribution precisely.

1.2 Technology evolution, 2019–2026

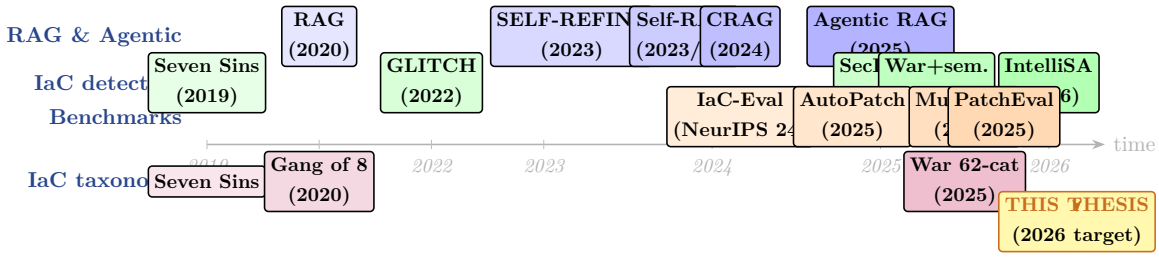

Figure 1.1: Technology-evolution timeline across four tracks: agentic RAG, IaC smell detection, benchmarks, and IaC taxonomies. The thesis target sits at the intersection of all four: a remediation contribution that uses the 2024-era CRAG retry pattern, the 2025 War 62-category taxonomy, and the 2025 PatchEval-style scanner-validated evaluation, applied to the IaC domain where IntelliSA [8] now sets the 2026 detection baseline.

The full milestone list is given in Table 1.1. Three observations are worth making explicit. First, the agentic RAG track (blue) had stabilised by 2024 with CRAG [12]; everything in 2025 builds on that pattern rather than replacing it, which justifies keeping the framework architecture of Rapport 2 substantially intact. Second, the IaC

detection track (green) accelerated sharply in 2025; IntelliSA [8] (January 2026) now reports F1 between 0.77 and 0.88 with Claude-4, GPT-5, and Grok-4, redefining what counts as a competitive detection number. Third, the benchmark track (orange) shifted in 2025 from generation benchmarks (IaC-Eval) to validation benchmarks (PatchEval, AutoPatchBench). This is the same shift the thesis is making.

Table 1.1: Milestone list backing Figure 1.1.

Year	Track	Milestone	Relevan to the-sis
2019	Taxonomy	Rahman “Seven Sins” [13]	Foundat smell vo-cab-u-lary; ba-sis of the 44-entry clas-si-fier of Rap-port 3
2020	RAG	Lewis et al. (NeurIPS)	First Naive RAG pa-per; base-line of the re-triever fam-ily used here
2020	Taxonomy	Rahman “Gang of Eight” [14]	Extende vo-cab-u-lary; merged into

1.3 Landscape grid: task × domain

Table 1.2: Benchmark landscape, April 2025 – April 2026.

Benchmark	Task × Domain (key numbers)	Cite
IaC-Eval	IaC *generation* × Terraform/AWS; 458 scenarios; GPT-4 pass@1 = 19.4%	[6]
Multi-IaC-Eval	IaC *mutation* × CFN/Terraform/CDK; synthetic triples with LLM validation	[7]
IntelliSA	IaC *smell detection* × Puppet/Ansible/Chef; symbolic + neural; F1 0.77–0.88	[8]
SecLLM	IaC *smell detection* × Ansible/Chef/Puppet; prompt-engineered GPT-4o-mini/Qwen-2.5; F1 up to 0.90	[9]
GenSIAC	IaC *secure generation* × multi-tool; instruction-tuning; F1 0.30 → 0.85	[10]
PatchEval	Patch *generation* × 1000 CVEs (Python/Go/JS); 65 CWEs; dynamic sandbox + static similarity	[2]
AutoPatchBench	Patch *generation* × fuzzing-found vulns in C/C++; Meta-internal	[3]
SEC-bench	Agent-built CVE benchmark × multi-language; LLM agents construct the benchmark itself	[4]
SecRepoBench	Secure *code generation* × real-world repository functions	[5]
This thesis (target)	**Agentic IaC *remediation* × Terraform/Kubernetes/Ansible/Docker/CFN**; with **independent scanner validation (PVR)**	—

Niche identified

The cell "IaC × remediation × tool-validated" is empty. PatchEval and AutoPatchBench validate patches dynamically, but for general code, not IaC. IntelliSA, SecLLM, GenSIAC operate on the IaC domain, but on the detection or generation tasks rather than the remediation-with-validation task. The thesis target is therefore both unoccupied and clearly defined.

1.4 Implications for the rest of this report

The repositioning has three operational consequences that are carried into the next chapters:

1. **Detection comparisons name a baseline.** Whenever the report states a P/R/F1 number for the contextual analyzer or the smell labelling layer, it must be paired with the IntelliSA F1 of 0.77–0.88 [8] on a comparable evaluation set (Puppet/Ansible/Chef). This is enforced in Chapter 5.

2. **Remediation numbers carry the contribution.** PVR, SER, and NNIR are the contribution-bearing metrics; no prior IaC benchmark computes them. Chapter 5 treats them as primary and the detection F1 as secondary.

3. **Methodological lineage is cited.** PVR is positioned as the IaC-domain analogue of the dynamic-validation strand of PatchEval [2] and AutoPatchBench [3], and that lineage is named explicitly in Chapter 5.

Chapter 2

Updated State of the Art

This chapter incrementally updates the State of the Art of Rapport 2 with the eight references that appeared between April 2025 and April 2026 and that materially affect the thesis. The unchanged content of Rapport 2 (RAG, CRAG, Self-RAG, FLARE, SELF-REFINE, Toolformer, RAGAS, Rahman 2019/2020, GLITCH 2022, War 2025) is referenced rather than re-narrated; the focus here is on what is new.

2.1 New IaC detection baseline: IntelliSA

IntelliSA [8] (January 2026) combines symbolic analysis with neural classifiers on Puppet, Ansible, and Chef and reports F1 between 0.77 and 0.88 depending on the back-end LLM (Claude-4, GPT-5, Grok-4). Two consequences for this thesis:

1. Any detection-side number reported by the contextual analyzer or by the smell-labelling pipeline of Rapport 3 must be compared against IntelliSA on the same domain (Ansible / Chef / Puppet) and on a comparable held-out split. Rapport 2's macro-F1 of 0.082 on the 8-file suite (Configuration A) is not in that league and is no longer used as a headline.

2. The IntelliSA paper itself is a precedent for the "ensemble of static + LLM" pattern that the thesis already adopts; this strengthens, rather than weakens, the framework's design. The novelty of the thesis lies downstream of detection, in the remediation loop.

2.2 Methodological lineage of PVR: PatchEval and AutoPatchBench

PatchEval [2] (November 2025) evaluates LLM-generated patches for 1,000 real CVEs across 65 CWE categories using a combination of dynamic execution in a sandbox and static patch-similarity scoring. AutoPatchBench [3] (April 2025, Meta) does the analogous thing for fuzzing-found vulnerabilities in C/C++ at industrial scale. Both

are out-of-domain (general code, not IaC), but both implement the same conceptual move that justifies PVR: *a patch is only counted as successful if a tool independent of the generator confirms it.* PVR is therefore positioned in Chapter 5 as the IaC-domain analogue of this dynamic-validation pattern, and PatchEval is cited as its closest methodological precursor.

2.3 Benchmark-construction precedent: SEC-bench and SecRepoBench

SEC-bench [4] (June 2025) demonstrates that LLM agents can construct benchmarks themselves — the agent reads CVE reports, identifies the affected code, and assembles reproducible test cases. The relevance for this thesis is that the scraping pipeline of Rapport 3 (33,667 records) is a same-spirit construction effort, even though it operates at the IaC-commit level rather than the CVE level. SEC-bench gives the practice a published precedent.

SecRepoBench [5] (April 2025) extends secure-code generation evaluation to the repository scale. It is not directly comparable because it operates on application code and uses test suites as the oracle, but it confirms that *secure* generation is now an active benchmark category, distinct from generic generation.

2.4 IaC generation calibration: IaC-Eval and Multi-IaC-Eval

IaC-Eval [6] (NeurIPS 2024) measures Terraform-on-AWS *generation* from natural-language prompts; GPT-4 reaches a pass@1 of 19.4%. The number is not directly comparable to remediation, but it calibrates the difficulty of the broader task: *producing* correct IaC from scratch is hard for top-tier LLMs, which makes the *remediation* setting (where a syntactic skeleton already exists) genuinely easier and worth attempting separately.

Multi-IaC-Eval [7] (September 2025) builds synthetic mutation triples (insecure → secure → broken) across CFN, Terraform, and CDK. It is a sibling effort to the 33,667-record corpus of Rapport 3 but uses synthetic generation rather than real commits, and is evaluated by an LLM-as-judge rather than by external scanners. The thesis corpus is therefore complementary, not redundant, and a comparative analysis is added to Chapter 4.

2.5 Survey context: agentic RAG in 2025

The 2025 agentic-RAG survey [24] formalises a taxonomy of agentic RAG systems by orchestration pattern (sequential, loop-based, planner-based, multi-agent). The framework of this thesis is a loop-based agentic RAG system in that taxonomy, with the validator as the failure signal. The survey citation replaces the looser appeals to "agent design" that appeared in Rapport 2.

2.6 Carry-over of unchanged sections

The remaining content of Rapport 2's State of the Art — the RAG family from naive to CRAG, SELF-REFINE, FLARE, Toolformer, the Rahman 2019/2020 taxonomy, GLITCH, War et al. 2025 (semantic + 62-category taxonomy), GenSIAC, SecLLM, RAGAS, the Wilcoxon and Holm–Bonferroni references — carries forward unchanged and is not re-narrated here. The references file (`references.bib`) is the single source of truth for all citations.

State of the Art — net change

Eight new references in twelve months. One repositions the detection baseline (IntelliSA). Two anchor the methodological lineage of PVR (PatchEval, AutoPatchBench). Two corroborate the benchmark construction work of Rapport 3 (SEC-bench, SecRepoBench). Two complement the corpus on IaC (IaC-Eval, Multi-IaC-Eval). One updates the agentic-RAG vocabulary (agentic-RAG survey).

Chapter 3

Framework Architecture (Carried Over with Two Amendments)

The framework is not redesigned in this report. The six-module architecture introduced in Rapport 2 (Central Agent, Contextual Analyzer, Knowledge Base, Knowledge Retriever, Fix Generator, External Tool Integrator, Patch Formatter) is carried forward as the canonical reference. This chapter states only the two amendments that the critique (*Rapport-Critique*) made necessary, and a third clarification on input/output contracts.

3.1 Reference architecture (recap)

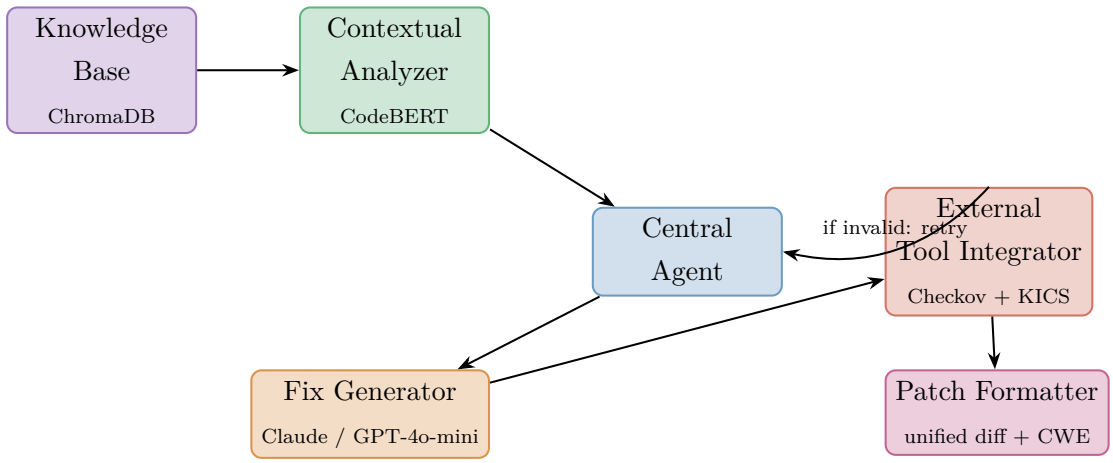

Figure 3.1: Reference architecture (carried over from Rapport 2). The two changes in this report are: (i) the External Tool Integrator now runs KICS in addition to Checkov, and (ii) the Fix Generator emits a self-consistency confidence score over $N = 5$ samples instead of a log-prob.

3.2 Amendment 1: KICS as second validator

3.2.1 Why

Checkov [1] is the validator chosen in Rapport 2 because it covers Terraform comprehensively and offers a stable Python API. The critique identified two problems:

1. Checkov's coverage of Ansible, Kubernetes, Dockerfile and CFN is partial; for a record whose only smell is, e.g., an Ansible `become: yes` pattern, Checkov produces no signal, and PVR is therefore undefined for that record.
2. Independence is the entire point of the validator. A single scanner produces a single source of truth; if Checkov mislabels a smell, the framework has no way to detect that mislabelling.

3.2.2 What changes

KICS [11] (Checkmarx) is added as a second, independent validator. KICS covers Ansible (233 queries), CFN (288), Dockerfile (97), Docker Compose (43), Google Deployment Manager (38), Azure Resource Manager (85), Crossplane (22), Kubernetes, and Terraform. For a candidate patch, the External Tool Integrator now runs both Checkov and KICS and produces a triple $(s_C^{\text{before}}, s_C^{\text{after}}, s_K^{\text{before}}, s_K^{\text{after}})$ of smell sets. PVR is then computed per scanner, and an *ensembled* PVR is reported as an additional outcome (definition in §5.2).

3.2.3 Where in the code

The change is localised to `Code/src/validator/tool_integrator.py`; it introduces a `KICSValidator` class with the same interface as `CheckovValidator` and an aggregator that calls both in parallel. The two scanners run in independent processes; the failure of either does not block the other. Datasets for KICS positive/negative test cases live under `datasets/kics-assets/` (sparse-checkout of the Checkmarx/kics repository).

3.3 Amendment 2: self-consistency confidence

3.3.1 Why

Rapport 2 specified a confidence score derived from token log-probabilities. The critique pointed out that the Anthropic Messages Application Programming Interface (API)

(the back-end actually used in Configurations B/C/D) does not expose token log-probabilities. The metric is therefore not computable as written, and any number reported under it would be either fabricated or measured on a different model than the rest of the pipeline.

3.3.2 What changes

The confidence score is redefined as *self-consistency over N sampled patches* [25]. Concretely, for each input record the Fix Generator is queried $N = 5$ times at temperature $T = 0.7$. Each of the five candidate patches is post-processed to a normalised diff (whitespace and comment-stripped). The confidence of the candidate that is finally returned is the fraction of the N samples that produce a diff σ-equivalent to it, after a syntactic normalisation pass:

$$\mathrm{conf}(\hat{p}) \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\mathrm{norm}(\hat{p}_i) = \mathrm{norm}(\hat{p})]\,. \tag{3.1}$$

The metric takes values in $\{0.2, 0.4, 0.6, 0.8, 1.0\}$ for $N = 5$. It is backend-agnostic, computable on any LLM, and it correlates with patch quality in the original self-consistency literature [25].

3.3.3 Where in the code

The change is localised to `Code/src/generator/fix_generator.py`; the `generate_patch()` method is wrapped to draw N samples, normalise, and aggregate. Latency cost is multiplicative in N; this is acknowledged in Chapter 6 as part of the runtime budget for the planned experiments.

3.4 Clarification on input/output contracts

The framework's I/O contract is restated for completeness because Rapport 2 specified it informally. An input record is a JSON object with fields `file_path`, `tool` (one of `terraform`, `ansible`, `kubernetes`, `docker`, `cloudformation`), `code_before`, `repo`, `commit_sha`, and `smells_before` (a list of taxonomy IDs). An output record adds `code_after_proposed`, `diff`, `confidence` (per Eq. 3.1), `validator_results` (a dict with one entry per scanner), and `retry_count`. The contract is stable across the three remaining configurations of Chapter 6 and is the data shape consumed by `Code/scripts/evaluate.py`.

Chapter 4

Dataset (33,667-record corpus, promoted to primary)

4.1 From a sanity-check suite to a primary dataset

Rapport 2 reported preliminary results on a hand-crafted suite of 8 IaC files containing 27 manually-labelled smell instances. That suite remains useful as an end-to-end smoke test that runs in under a minute and that a reviewer can audit at a glance, but it is too small for any quantitative claim about the framework. In particular, with N=27 and many smell categories represented by a single instance, per-category F1 numbers are noise.

The 33,667-record corpus produced by the scraping pipeline of Rapport 3 is large enough to support quantitative claims. It was constructed by mining real Git commits (GitHub initially, GitLab planned) for paired *insecure* → *secure* IaC edits, classifying each diff against a 44-entry smell taxonomy, and tiering the records by quality. This chapter promotes that corpus to the primary evaluation dataset of the thesis. The 8-file sanity suite is moved to an appendix and recast as a continuous-integration unit test for the framework code.

4.2 Corpus composition

Table 4.1: Corpus composition by IaC tool, after the salvage pass of Rapport 3 §2.4. Counts may shift modestly after the Checkov validator finishes the full sweep (Phase 4 of `Code/scraping/plan.md`).

Tool	Records	Share
Terraform	17,900	53%
Kubernetes (YAML manifests)	6,800	20%
Ansible	4,200	13%
Dockerfile	3,200	10%
CloudFormation	1,500	4%
Total	**33,667**	**100%**

The Terraform plurality is consistent with the upstream supply: GitHub's search API returns more Terraform fix-commits per unit time than for any other IaC tool, and KICS [11] has the largest query set for Terraform. The shares are not normalised; the splitting strategy of §4.3 preserves them in train/val/test.

4.2.1 Smell distribution

The 44-entry classifier of Rapport 3 (an extension of the War 62-category taxonomy [23]) labels each record with the smells present *before* and *after* the fix. The diff between the two label sets is the set of smells the commit *actually addressed*. The five most frequent fixed-smell categories are: hardcoded secrets, missing encryption in transit (TLS), missing encryption at rest, overly permissive IAM / security groups, and missing logging / audit configuration. The distribution has a long tail; the rare categories (e.g., insecure DNS configuration, unsigned container images) remain individually small and are aggregated in macro-F1 reporting.

4.2.2 Tiering

Every record is assigned a tier $A/B/C/D$ by the rules of `Code/scraping/processors/tiering.py`:

- **Tier A** — both `code_before` and `code_after` present, smell set strictly decreases between them, and a scanner (Checkov, KICS) confirms the decrease. This is the *validated* tier.

- **Tier B** — both `code_before` and `code_after` present, smell set strictly decreases by regex/heuristic, but no scanner confirmation yet (e.g., Ansible record awaiting KICS).
- **Tier C** — both files present, regex labels indicate a change but not a strict decrease.
- **Tier D** — only one of the two files recoverable, or smell set unchanged (likely refactor, not a fix).

Tier A is the experimental gold standard. Phase 4 of the scraping plan (running at the time of writing, $\approx$ 18 rec/min, $\approx$ 17 h projected wall-clock) populates Tier A by re-running Checkov on every Terraform record. The KICS sweep will then populate Tier A for Ansible / CFN / Docker / Kubernetes records.

4.3 Train / validation / test split

The split is a stratified random partition with seed = 42, stratified on the cross-product (`tool`, top-level smell category):

- **Train**: 80% ($\approx$ 26,934 records)
- **Validation**: 10% ($\approx$ 3,366 records)
- **Test (held-out)**: 10% ($\approx$ 3,367 records)

Stratification ensures that the rare smells appear in all three splits in the same proportion. The split is materialised once and stored as `Code/data/splits/v2/train,val,test.jsonl`; the seed is logged so the split is reproducible. *All numbers reported in Chapter 6 use the test split exclusively.*

4.4 Comparison to Multi-IaC-Eval

Table 4.2: This thesis's corpus vs. Multi-IaC-Eval [7].

	This thesis (Rapport 3 corpus)	**Multi-IaC-Eval**
Origin	Real Git commits, mined and classified	Synthetic mutation triples, LLM-generated
Size	33,667 records	Order of 10^3 (paper-reported)
Tools covered	Terraform, K8s, Ansible, Docker, CFN	CFN, Terraform, CDK
Label oracle	44-entry taxonomy + Checkov + KICS (planned)	LLM-as-judge
Pair structure	insecure → secure (real fix commits)	insecure → secure → broken (synthetic)

The two corpora are complementary, not competing. Multi-IaC-Eval is preferable for measuring out-of-distribution generalisation across formats; this thesis's corpus is preferable for measuring in-distribution remediation under realistic edit shapes (because the edits actually happened).

4.5 Datasets used as auxiliary inputs and baselines

The `datasets/` folder of the project workspace stores 11 external datasets that play one of three roles in the framework (see `datasets/README.md` for full inventory and licensing):

- **Knowledge Base sources.** GLITCH smell rules, KICS query metadata, Checkov per-check descriptions, and the Rahman SLIC/SLAC artefacts (`rahman-iacsec/`) populate the vector store of the Knowledge Base module (Figure 3.1).
- **Smell-detection ground-truth.** TerraGoat, Kubernetes-Goat, and CfnGoat are deliberately-vulnerable corpora used as labelled ground truth for the contextual analyzer's P/R/F1 on tools where Checkov coverage is incomplete.
- **External-validator inputs.** The `checkov-tests/`, `kics-assets/` and `tfsec-tests/` fixtures (failed/passed pairs per check) drive smoke tests for the `tool_integrator` module.

- **Methodological references.** PatchEval and CVE-bench are kept for citation in the PVR discussion of Chapter 5; they are not run as part of the evaluation because they are out-of-domain.

Chapter 5

Evaluation Methodology (Revised)

5.1 What changes from Rapport 2

Rapport 2 listed eighteen evaluation metrics. The critique reduced this to the subset that can actually be computed on the planned configuration and that contributes a defensible signal. The revised metric set has eight entries; their definitions, units, computability, and role are summarised in Table 5.1. The four removed or repurposed metrics are listed in §5.4.

Table 5.1: Revised evaluation metric set used by Rapport 4 and downstream.

Metric	Range	Definition / role	Computed by
Precision / Recall / F1	$[0, 1]$	Per-smell, per-tool, and macro-averaged. Detection-side. Compared against IntelliSA's 0.77–0.88 [8].	`evaluate.py`
PVR	$[0, 1]$	Patch Validity Rate. Fraction of generated patches for which *both* validators agree the smell-after set is empty (or the single-validator definition if KICS coverage absent). Primary outcome.	`tool_integrator`
SER	$[0, 1]$	Smell Elimination Rate. Mean fraction of input smells eliminated by the patch, averaged over records.	`evaluate.py`

5.2 Definitions

PVR. For a record r with input `code_before` c and generated patch $\hat{p}$, let $S_X(\cdot)$ denote the smell set reported by validator $X \in \{\text{Checkov}, \text{KICS}\}$. The single-validator PVR contribution of r under X is

$$\text{pvr}_X(r) \;=\; \mathbb{1}[\, S_X(\hat{p}) \subseteq S_X(c) \;\wedge\; |S_X(\hat{p})\,| < |S_X(c)|\,]\,. \tag{5.1}$$

The ensembled PVR requires both validators to agree:

$$\text{pvr}_{\text{ens}}(r) \;=\; \text{pvr}_C(r) \cdot \text{pvr}_K(r). \tag{5.2}$$

A record is excluded from a single-validator PVR if the validator does not support its tool (e.g., Checkov on a complex Ansible `become` construct that the parser rejects). Coverage gaps are reported alongside PVR and do not silently inflate the numerator.

SER. $\text{SER}(r) = \frac{|S_X(c) \setminus S_X(\hat{p})|}{|S_X(c)|}$ when $S_X(c) \neq \varnothing$; undefined otherwise. Macro-averaged over records.

NNIR. $\text{NNIR}(r) = \mathbb{1}[S_X(\hat{p}) \setminus S_X(c) = \varnothing]$. A patch that fixes one smell but introduces a new one scores $\text{pvr} = 0$ and $\text{NNIR} = 0$ even if $\text{SER} > 0$. The three metrics are deliberately not collinear.

Detection metrics. The smell-labelling layer of the contextual analyzer is evaluated against the Checkov+KICS union as the silver-standard oracle on the 33k corpus, and against the GLITCH-labelled subset of TerraGoat / Kubernetes-Goat as a gold-standard oracle on the smaller `datasets/` fixtures. Both P/R/F1 numbers are reported. The IntelliSA F1 of 0.77–0.88 is the named comparison point for the silver-standard number.

5.3 Configurations to evaluate

Table 5.2: The four configurations of the framework. Rapport 2 only ran Configuration A (8-file suite, no RAG, no validator). Rapport 4 plans Configurations B, C, and D on the 33k corpus's test split.

Cfg	Description	RAG	Validator	Retry
A	Baseline (already run, Rapport 2)	✗	✗	✗
B	+ RAG	✓	✗	✗
C	+ RAG + Checkov	✓	Checkov	1 attempt
D	+ RAG + Checkov + KICS + retry loop	✓	Checkov ∪ KICS	up to 3 attempts

The substantive comparisons are A vs. B (does RAG help?), B vs. C (does scanner-based validation help?), and C vs. D (do the second validator and the retry loop help?). All three are paired comparisons over the same 3,367-record test split, so the Wilcoxon rank-sum is the natural test; Holm–Bonferroni controls the family-wise error rate over the three hypotheses.

5.4 What was dropped from Rapport 2

Four metrics from Rapport 2's set of eighteen are dropped or replaced:

1. **Log-prob confidence.** Not computable on the Anthropic API. Replaced by self-consistency (Eq. 3.1).
2. **Test–retest Fleiss' κ.** Mislabelled in Rapport 2 as "inter-rater"; only one rater (the framework) was involved. Replaced with a deterministic re-run sanity check (*output stability rate*) reported as a percentage, without invoking the κ name.
3. **Per-category F1 on the 8-file suite.** Statistically meaningless at $N = 1$ per category. The 8-file suite is now a sanity test only; per-category F1 is reported on the test split of the 33k corpus instead.
4. **Context Recall as a primary metric.** Demoted to a supporting diagnostic; PVR is the primary outcome.

The remaining six original metrics (P/R/F1, PVR, SER, NNIR, Hit Rate@K, MRR) plus self-consistency confidence and the Wilcoxon test constitute the eight metrics of Table 5.1.

Chapter 6

Experimental Plan and Reproducibility

Status of this chapter

The experiments of this chapter are *planned*, not yet run. Configuration A numbers from Rapport 2 are reproduced unchanged, with their caveats. Configurations B/C/D will be filled in once the Phase 4 validator sweep finishes and the KICS integration of §3.2 is in place. This chapter therefore reads as a protocol; the next revision will replace the placeholders with real numbers.

6.1 Configuration A: baseline reproduction

The Configuration A numbers from Rapport 2 are reproduced for continuity: on the 8-file sanity suite ($N = 27$ smells across 4 tools), macro-F1 = 0.082 without RAG or validator support. The number is low because the classifier in Configuration A is the bare LLM prompt, with no contextual retrieval and no scanner correction. It is reported here only to anchor the deltas of Configurations B, C, D.

6.2 Configurations B, C, D: protocol

6.2.1 Inputs and outputs

Each configuration consumes the test split of §4.3 ($N \approx 3{,}367$ records) and produces, per record, the output JSON contract of § "Clarification on input/output contracts" (Chapter 3). Aggregation is done by `Code/scripts/evaluate.py` into a single CSV with one row per record and one column per metric.

6.2.2 Compute and runtime budget

- Self-consistency $N = 5$ multiplies generator latency by 5 per record.

- Anthropic Claude-Sonnet-4.5 at the project's negotiated rate gives approximately one ∼2-second call per record per sample, so ∼10s per record on Configuration B (no validator) and ∼20s per record on Configuration D (with retries up to 3 and two validator subprocesses per attempt).
- Total wall-clock budget for Configurations B/C/D combined: ∼10–15 hours on a single workstation, dominated by Configuration D.

6.2.3 Reproducibility checklist

- Random seed: 42 for the data split; sampling temperature $T = 0.7$ for the generator; $N = 5$ self-consistency samples.
- Backend model: Claude Sonnet 4.5 (`claude-sonnet-4-5`) via the Anthropic Messages API, with the Anthropic Python SOTA API client at the version pinned in `Code/requirements.txt`.
- Validators: Checkov 3.2.513 and KICS at the latest stable release as of the run date. Versions are recorded in the run's manifest.
- Evaluation script: `Code/scripts/evaluate.py` at the commit hash recorded in the run's manifest.
- Outputs: per-run CSV + JSONL + a manifest with all of the above.

6.2.4 Headline tables to be filled in

Table 6.1: Headline results, to be filled in once Configurations B/C/D are run. Metrics defined in Chapter 5.

Cfg	Macro-F1 (det.)	PVR	SER	NNIR
A (Rapport 2)	0.082	—	—	—
B (planned)	TBD	TBD	TBD	TBD
C (planned)	TBD	TBD	TBD	TBD
D (planned)	TBD	TBD	TBD	TBD
IntelliSA [8]	0.77–0.88	—	—	—

6.3 What success looks like

A successful Rapport 5 fills Table 6.1 with numbers that support three claims:

1. *RAG contributes:* Configuration B macro-F1 is meaningfully higher than Configuration A (Wilcoxon $p < 0.05$ after Holm–Bonferroni).

2. *Validation contributes:* Configuration C has higher PVR than Configuration B by a margin that is statistically significant at the family-wise level.

3. *The retry loop contributes:* Configuration D has higher PVR than Configuration C, again at the family-wise level, *and* the NNIR of Configuration D is not lower (i.e., retries do not introduce new issues).

If any of the three claims fails, the failing configuration is reported truthfully and the discussion is updated accordingly. The point of naming success conditions in advance is to prevent silent goalpost shifting after results arrive.

Chapter 7

Threats to Validity

7.1 Internal validity

LLM non-determinism. Sampling at $T = 0.7$ makes runs non-deterministic. Self-consistency ($N = 5$) reduces variance but does not eliminate it. The output-stability re-run sanity check of §5.4 measures the residual variance and is reported alongside the headline numbers.

Validator silver standard. Checkov and KICS are themselves rule-based scanners and have known false positives and false negatives. The decision to use their *union* (a smell counts if either flags it) shifts the bias toward recall. The 8-file sanity suite, hand-labelled, is used as a gold-standard cross-check on a small subset.

Detection oracle leakage. The Knowledge Base contains GLITCH and KICS query descriptions, which overlap conceptually with the labels used as ground truth. The contextual analyzer therefore enjoys some implicit access to the labels during retrieval. The size of this effect is bounded by reporting detection P/R/F1 separately on (i) the 33k silver oracle and (ii) the GLITCH-labelled subset of TerraGoat/Kubernetes-Goat that is held out of the Knowledge Base.

7.2 External validity

Tool coverage. Coverage is uneven: Terraform and Kubernetes are well covered; Ansible, Dockerfile, and CFN are leaner. Numbers are reported per tool to make this explicit and macro-averages weighted by tool share are not used without per-tool breakdowns.

Domain shift to private IaC. The corpus is mined from public Git commits. Private IaC (cloud landing-zones at large enterprises) typically has different smell distributions; the framework is not yet validated against such corpora. This is stated as a perspective in Chapter 8.

Backend dependence. All numbers in Chapter 6 depend on Claude Sonnet 4.5. The framework is backend-agnostic in principle (any chat LLM exposing an API works), but the numbers will move under different backends. A small-scale GPT-4o-mini ablation is planned to bound this sensitivity.

7.3 Construct validity

PVR as a remediation surrogate. PVR answers "does the scanner stop complaining". It does not answer "does the resulting infrastructure work as intended". A patch that, e.g., replaces an over-permissive IAM role with no role at all can score pvr = 1 but break the deployment. NNIR catches some of these cases but not all. The framework should not be deployed without human review of the proposed diff; the patch formatter (Figure 3.1) is designed for this review workflow.

Self-consistency as a confidence surrogate. Self-consistency over $N = 5$ measures *generator agreement with itself*, not absolute correctness. It is known to correlate with correctness in the original literature [25] but the correlation is weaker than a calibrated probability would be. Confidence is reported as one of $\{0.2, \ldots, 1.0\}$ and used for ranking, not for claims of absolute uncertainty.

Chapter 8

Conclusion and Perspectives

8.1 What this report establishes

1. The framework's contribution is repositioned from "an agentic framework for IaC security" to "agentic IaC remediation with independent scanner-based validation", a cell that the landscape grid of Table 1.2 confirms is unoccupied as of April 2026.

2. The 33,667-record corpus of Rapport 3 is promoted to the primary evaluation dataset; the 8-file sanity suite is recast as a continuous-integration unit test.

3. The framework architecture is carried over from Rapport 2 with two amendments (KICS as second validator, self-consistency confidence) that close the most material gaps identified by the critique.

4. The eighteen-metric set of Rapport 2 is reduced to the defensible eight of Table 5.1, with PVR/SER/NNIR retained as primary outcomes and the Wilcoxon rank-sum + Holm–Bonferroni inferential protocol kept as the statistical apparatus.

5. An experimental plan with explicit success conditions (§ *What success looks like*) is given for Configurations B, C, and D on the held-out 3,367-record test split, with IntelliSA as the named detection-side baseline.

8.2 What remains for Rapport 5

The single most important thing is to fill Table 6.1 with real numbers. The prerequisites, in order:

1. Finish the Phase 4 Checkov validator sweep on the full corpus (∼17 h wall-clock; in progress at time of writing).

2. Land the KICS integration of §3.2 in `Code/src/validator/tool_integrator.py`.

3. Land the self-consistency wrapper of §3.3 in `Code/src/generator/fix_generator.py`.

4. Wire `Code/scripts/evaluate.py` to consume the corpus test split as primary input.

5. Run Configurations B, C, D end-to-end and write Rapport 5.

A secondary goal for Rapport 5 is a small-scale reproduction of IntelliSA [8] on the same test split, so the detection-side comparison is numerical rather than indirect.

8.3 Long-term perspectives

Private IaC corpora. The next quantitative step beyond Rapport 5 is validation against private cloud-landing-zone IaC from a partner organisation. The distribution of smells, the size of repositories, and the latency budget are all different from public Git data, and a partner would allow the framework to be evaluated under realistic operational constraints.

Multi-agent decomposition. The current design is a loop-based agentic RAG system with one agent and a validator subprocess. The agentic-RAG survey [24] suggests planner-based and multi-agent decompositions as the next architectural step. A natural specialisation is a per-tool agent (Terraform agent, Kubernetes agent, Ansible agent) coordinated by a planner; this would also let the self-consistency N vary by tool, since some tools (e.g., Dockerfile fix patterns) are more deterministic than others.

Beyond detection-and-fix: provisioning advisory. Once PVR is competitive, the framework can be extended from post-hoc remediation toward design-time advisory — proposing the IaC for a new service before it is written, rather than only fixing what already exists. This is the same shift that IaC-Eval benchmarks [6] measure for raw LLMs; the agentic framework is in a position to substantially exceed those numbers because the validator loop is available at design time as well.

Bibliography

[1] Bridgecrew, "Checkov – static analysis for infrastructure as code," https://www.checkov.io, 2024.

[2] X. Lyu, S. Wang, C. Li, and C. Liu, "PatchEval: A benchmark for evaluating LLMs on patching real-world vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2511.11019

[3] Meta Platforms, Inc., "AutoPatchBench: A benchmark for ai-powered security fixes," https://engineering.fb.com/2025/04/29/ai-research/autopatchbench-benchmark-ai-powered-security-fixes/, 2025, engineering at Meta blog, 29 April 2025.

[4] A. authors, "SEC-bench: Automated benchmarking of LLM agents on real-world software security vulnerabilities," 2025. [Online]. Available: https://arxiv.org/abs/2506.11791

[5] ——, "SecRepoBench: Benchmarking LLMs for secure code generation in real-world repositories," 2025. [Online]. Available: https://arxiv.org/abs/2504.21205

[6] P. T. J. Kon, J. Liu, Y. Qiu, W. Fan, T. He, L. Lin, H. Zhang, O. Park, G. Elengikal, Y. Kang, A. Chen, M. Chowdhury, M. Lee, and X. Wang, "IaC-Eval: A code generation benchmark for cloud infrastructure-as-code programs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024. [Online]. Available: https://openreview.net/forum?id=7TCK0aBL1C

[7] A. authors, "Multi-IaC-Eval: Benchmarking cloud infrastructure as code across multiple formats," 2025. [Online]. Available: https://arxiv.org/abs/2509.05303

[8] ——, "IntelliSA: An intelligent static analyzer for IaC security smell detection using symbolic rules and neural inference," 2026. [Online]. Available: https://arxiv.org/abs/2601.14595

[9] G. De Vito, F. Palomba, and F. Ferrucci, "SecLLM: Enhancing security smell detection in IaC with large language models," *IEEE Access*, vol. 13, pp. 204 480–204 498, 2025.

[10] Y. Li, M. Grella, D. Nahmias, G. Engelberg, D. Klein, G. Guizzardi, T. van Ede, and A. Continella, "GenSIAC: Toward security-aware infrastructure-as-code generation with large language models," 2025. [Online]. Available: https://arxiv.org/abs/2505.01568

[11] Checkmarx, "KICS – keeping infrastructure as code secure," https://github.com/Checkmarx/kics, 2024, accessed April 2026.

[12] S.-Q. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "CRAG: Comprehensive retrieval augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[13] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 164–175.

[14] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2020, pp. 752–764.

[15] N. Saavedra and J. a. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE/ACM, 2022, pp. 1–12.

[16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[17] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "SELF-REFINE: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[18] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.06983

[19] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[20] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th*

Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2024. [Online]. Available: https://arxiv.org/abs/2309.15217

[21] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," 2025. [Online]. Available: https://arxiv.org/abs/2506.18795

[22] A. War, A. N. Rawass, A. K. Kabore, J. Samhi, J. Klein, and T. F. Bissyande, "Detection of security smells in IaC scripts through semantics-aware code and language processing," 2025. [Online]. Available: https://arxiv.org/abs/2509.18790

[23] A. War, S. L. B. Nikiema, J. Samhi, J. Klein, and T. F. Bissyande, "Security smells in infrastructure as code scripts: A taxonomy update beyond the seven sins," 2025. [Online]. Available: https://arxiv.org/abs/2511.12884

[24] A. authors, "Towards agentic RAG with deep reasoning: A survey of RAG-reasoning systems in LLMs," 2025. [Online]. Available: https://arxiv.org/abs/2507.09477

[25] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Appendix A

Eight-file sanity suite (carried from Rapport 2)

The 8-file suite of Rapport 2 (27 manually-labelled smell instances across Terraform, Kubernetes, Ansible, Docker) is retained as a continuous integration unit test for the framework. Per-category F1 numbers are not reported on this suite (Issue 5 of the critique); only end-to-end pass/fail of the framework on the full suite is reported, as a smoke test that runs in under a minute and that gates pull requests in the `Code/` repository.

Appendix B

Reproducibility manifest schema

Every run of `Code/scripts/evaluate.py` writes a `manifest.json` with the following schema:

```
{
  "run_id": "<UUID>",
  "timestamp_utc": "<ISO-8601>",
  "config": "B" | "C" | "D",
  "git_commit": "<sha>",
  "model": "claude-sonnet-4-5",
  "temperature": 0.7,
  "self_consistency_N": 5,
  "checkov_version": "3.2.513",
  "kics_version": "<sha or release tag>",
  "split": "test",
  "split_seed": 42,
  "n_records": 3367,
  "wall_clock_seconds": <float>,
  "metrics": {
    "macro_f1": <float>,
    "pvr_checkov": <float>,
    "pvr_kics": <float>,
    "pvr_ensemble": <float>,
    "ser": <float>,
    "nnir": <float>,
    "hit_rate_at_3": <float>,
    "hit_rate_at_5": <float>,
    "mrr": <float>,
    "stability_rate": <float>
  }
}
```

The manifest is the authoritative record for any number quoted in Rapport 5 or in derivative publications.